Assignment: Ingest, Store, Visualize, and Answer Questions with RAG

Overview

This assignment mirrors a realistic slice of our stack: pulling data from an external source, turning it into a clean, queryable dataset, exposing it through an API, and layering a Retrieval-Augmented Generation (RAG) pipeline on top so the data can be queried in natural language. We care less about a polished UI and more about how you structure data, design APIs, and reason about an LLM-backed system end to end.

You will build a small pipeline with five parts:

- Data ingestion: download a dataset from a live public API
- Preprocessing: clean and normalize it into a well-structured form
- Storage: persist it in a database with a sensible schema
- Visualization API: one API that serves aggregated data for charting, backed by the DB
- RAG pipeline: embed the unstructured fields, retrieve relevant records, and answer free-text questions about the dataset through an API

The Dataset: arXiv

Use the arXiv API (export.arxiv.org/api/query) as your data source. No signup, no API key, no account — it's a plain HTTP GET that returns an Atom/XML feed of papers. It gives you both structured fields (good for the visualization API) and a substantial abstract per paper (good for RAG) from a single source.

Pull at least 1,000 papers (more is fine) from one or more categories (e.g. cs.AI, cs.LG, cs.CL), paginating with the start / max_results parameters, and keep at least:

- Structured fields: arxiv_id, title, authors, primary_category, all categories, published date, updated date, and (when present) doi / journal_ref
- Unstructured fields: the abstract (the <summary> element), this is your main text for RAG

Requirements

Data Ingestion

- Fetch data from the arXiv API on demand (a runnable script or CLI command, not a one-off notebook).
- Handle pagination (start / max_results) and self-throttle requests (arXiv asks for ≤1 request per 3 seconds).

- Make re-running ingestion safe, re-fetching shouldn't duplicate rows or crash on partial data.
- Support updating the dataset: re-running ingestion should pick up new and changed papers from the arXiv API and refresh the RAG index (embeddings) so it reflects the updated data.
- Provide a way to wipe the current dataset clean (papers, authors, categories, and embeddings) so a fresh import can be run from scratch.

Preprocessing

- Parse the Atom/XML response into clean records; normalize dates and handle papers with missing doi/journal_ref.
- Flatten nested fields you need (authors, categories) into a form your schema can store and query — a paper can have multiple authors and multiple categories.
- Light text cleanup on the abstract.

Storage

- Persist the cleaned data in a relational database with a schema you design (papers, authors, categories as separate tables/join tables where it makes sense).
- Include migrations so the schema is reproducible from scratch.

Visualization API

Build one API that returns aggregated data computed from the database, not recomputed from the raw source, for charting. Pick at least 3 aggregations, for example:

- Paper count by category over time (e.g. per month or year)
- Top N categories by number of papers
- Top N authors by number of papers, or average number of authors per paper

RAG Pipeline

Build a pipeline that lets someone ask natural-language questions about the paper dataset and get a grounded answer:

- Generate embeddings for the unstructured field (the abstract, optionally concatenated with the title) and store the vectors (pgvector, Chroma, FAISS, or similar).
- On a query, retrieve the most relevant papers and pass them as context to an LLM to produce an answer.
- Expose it as an API, e.g. POST /ask {"question": "..."} -> {"answer": "...", "sources": [...]}, where sources identifies which papers grounded the answer.
- Handle the no-good-match case honestly rather than hallucinating an answer.

Your pipeline should reasonably answer questions ask by the user related to the papers.

Non-Functional Expectations

- Clear README: setup, environment variables, how to run ingestion, how to start the API, sample requests/responses for both APIs.
- docker-compose up should bring up the database and the API with no manual steps beyond setting a .env.
- Reasonable error handling and logging.
- A handful of automated tests.
- Commit history that shows incremental progress rather than a single commit.

Deliverables

- Create a **private** Git repository, push all of your work to it, and grant access to the GitHub account **@PipelineLab0** (github.com/PipelineLab0). Add it as a collaborator on the repository — your submission can't be reviewed until that access is granted.
- docker-compose.yml (or equivalent) to run the stack locally.
- .env.example listing required environment variables (API keys, DB URL, etc.) — never commit real keys.
- README.md covering setup/run instructions, your schema and architecture at a high level, sample curl requests and responses, known limitations, and what you'd improve with more time.

Timeline

You have 4 days from receiving this email to submit. In terms of focused effort, most candidates spend roughly 8–15 hours total.